Cletus — Section 1, Day 1

- **Date:** [YYYY-MM-DD]
- **Course:** Week 1, Day 1 — Running your first LLM locally, environment setup, website summarizer
- **Lab notebook:** week1/day1.ipynb (free Ollama version: week1/solutions/day1_with_ollama.ipynb)
- **Time spent:** [e.g. 2h]

Key concepts

- **LLM** — a model that predicts the next piece of text (token) from the text so far.
- **Ollama** — runs open-source models free on your own machine: `ollama run llama3.2` (use `llama3.2:1b` on small machines; avoid `llama3.3`, at 70B it's too big).
- **Frontier model** — the biggest, strongest closed models (GPT, Claude, Gemini), called over an API.
- **API key + `.env`** — keys live in a `.env` file (`OPENAI_API_KEY=sk-proj-...`) and are loaded with `load_dotenv()` . Never commit it.
- **System prompt** — sets the task, role and tone, e.g. "You are a snarky assistant that summarizes websites".
- **User prompt** — the actual request and content the model should respond to.
- **Messages** — the API takes a list of `{"role": ..., "content": ...}` dicts with roles `system` and `user` (later also `assistant`).
- **Chat Completions API** — send messages, get the model's reply back in `response.choices[0].message.content` .
- **Summarization** — a classic GenAI business use case: news, reports, CVs, reviews, meeting notes.

Code / examples

The whole Day 1 project in a few lines:

```
from dotenv import load_dotenv
from openai import OpenAI
from scraper import fetch_website_contents  # requests + BeautifulSoup: title + text, first 2,000
chars

load_dotenv(override=True)
openai = OpenAI()  # reads OPENAI_API_KEY from the environment

system_prompt = "You summarize websites in markdown, ignoring navigation text."
user_prompt_prefix = "Here are the contents of a website. Provide a short summary:\n\n"

def summarize(url):
    website = fetch_website_contents(url)
    messages = [
        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt_prefix + website},
    ]
    response = openai.chat.completions.create(model="gpt-4.1-mini", messages=messages)
    return response.choices[0].message.content
```

```
print(summarize("https://edwarddonner.com"))
```

Flow: **scrape page → build messages → call model → read `choices[0].message.content`**

Exercise / challenge

1. Change the tone in the system prompt ("respond in Spanish", "be formal", "be funny") and compare the summaries.
2. Make your own summarizer without a website: paste an email or article into the user prompt and ask for a 3-bullet summary plus a suggested subject line.
3. Run `summarize("https://openai.com")`. It returns almost nothing. Why? (See below.)

Questions & blockers

Watch out for: - Sites built with JavaScript (React apps) or protected against bots give little or no text, because the simple scraper only reads raw HTML. Selenium or Playwright can fix this. - `NameError` in a notebook → run the cells from the top down. Also select the `.venv` kernel in every notebook. - OpenAI keys start with `sk-proj-` and must have no spaces around them in `.env`.

To discuss with my partner

- Where could we use summarization at work?
- Which setup problems did we each hit (Cursor, uv, kernel, API key)?

Resources

- Week 1 code: https://github.com/ed-donner/llm_engineering/tree/main/week1
- Setup: `setup/SETUP-new.md`; fixes: `setup/troubleshooting.ipynb`
- Free alternative to paid APIs: `guides/09_ai_apis_and_ollama.ipynb`